

1 ЦЕЛЬ И ЗАДАЧИ ПРАКТИЧЕСКОЙ РАБОТЫ

Цель работы: освоение программного управления физическими процессами в **Blender** с помощью встроенного **Python API** (модуля `bpy`), включая автоматическую генерацию объектов, назначение физических параметров и запуск симуляции.

Задачи:

1. Изучить структуру Python API Blender для работы с физикой (`bpy.ops.rigidbody`, `bpy.data.objects`).
2. Научиться программно создавать сцену с множеством однотипных объектов – костяшек домино.
3. Освоить назначение физических параметров объектов: массы, трения, упругости и типа твёрдого тела – с помощью скрипта.
4. Реализовать анимацию объекта-толкателя с использованием Python.
5. Выполнить симуляцию и зафиксировать её результаты: время падения и количество упавших костяшек.
6. Проанализировать влияние параметров физики на поведение системы.

2 ТЕОРЕТИКО-МЕТОДИЧЕСКОЕ ВВЕДЕНИЕ

2.1 Blender предоставляет полнофункциональный Python API (`bpy`), позволяющий автоматизировать различные действия: от создания объектов до настройки физических свойств. Доступ к параметрам физики осуществляется через свойство `rigid_body` объекта.

2.2 Основные элементы Python API для работы с физикой:

- `bpy.ops.rigidbody.object_add(type='ACTIVE')` – добавление активного твёрдого тела;
- `bpy.ops.rigidbody.object_add(type='PASSIVE')` – добавление пассивного твёрдого тела;
- `obj.rigid_body.mass` – масса объекта в килограммах;
- `obj.rigid_body.friction` – коэффициент трения;
- `obj.rigid_body.restitution` – коэффициент упругости;
- `obj.rigid_body.collision_shape` – форма коллизии ('BOX', 'MESH', 'SPHERE' и др.).

2.3 Для программного перемещения объектов используются свойства `obj.location = (x, y, z)` и `obj.rotation_euler = (rx, ry, rz)`. Ключевые кадры

добавляются с помощью метода `obj.keyframe_insert(data_path="location", frame=N)`.

2.4 Для запекания симуляции физики используется команда `bpy.ops.ptcache.bake_all(bake=True)`. Запустить воспроизведение анимации можно с помощью команды `bpy.ops.screen.animation_play()`.

2.5 Для анализа результатов можно использовать свойство `bpy.context.scene.frame_current`, которое содержит номер текущего кадра. При частоте 24 кадра в секунду время симуляции определяется по формуле: $t = \frac{N}{24}$ где N – номер кадра.

3 ПРАКТИЧЕСКОЕ ЗАДАНИЕ: «КАСКАДНОЕ ПАДЕНИЕ ДОМИНО»

Описание задания: создайте сцену, в которой 30 костяшек домино выстроены в линию. Первая костяшка должна получить импульс от объекта-толкателя, запускающего цепную реакцию падения.

3.1 Откройте Blender и удалите стандартный куб: X → Delete.

3.2 Откройте рабочее пространство для написания скриптов: выберите вкладку Scripting на верхней панели интерфейса.

3.3 Нажмите кнопку New («Новый») для создания нового скрипта. Назовите файл domino_setup.py.

3.4 Вставьте в редактор скриптов следующий код:

```
import bpy

ver = bpy.app.version

print(f"Blender: {ver[0]}.{ver[1]}.{ver[2]}")

bpy.ops.object.select_all(action='SELECT')
bpy.ops.object.delete(use_global=False)

# Очистка данных
for block in [bpy.data.meshes, bpy.data.materials, bpy.data.textures,
              bpy.data.images, bpy.data.curves, bpy.data.lights]:
    for item in block:
        block.remove(item)

# Сброс кэша физики
bpy.ops.ptcache.free_bake_all()

scene = bpy.context.scene

if scene.rigidbody_world is None:
    bpy.ops.rigidbody.world_add()

rbw = scene.rigidbody_world
rbw.enabled = True

domino_count = 30
spacing = 0.65
domino_size = (0.1, 0.4, 0.8)

bpy.ops.mesh.primitive_plane_add(size=30, location=(0, 0, 0))
floor = bpy.context.active_object
floor.name = "Floor"
floor.location = (0, 0, 0)
```

```

floor.rotation_euler = (0, 0, 0)
floor.scale = (1, 1, 1)

bpy.ops.object.transform_apply(location=True, rotation=True, scale=True)
bpy.ops.rigidbody.object_add(type='PASSIVE')

floor.rigid_body.kinematic = True
floor.rigid_body.collision_shape = 'BOX'
floor.rigid_body.friction = 0.5
floor.rigid_body.restitution = 0.1
floor.rigid_body.use_margin = True
floor.rigid_body.collision_margin = 0.001

for i in range(domino_count):
    x_pos = i * spacing

    bpy.ops.mesh.primitive_cube_add(
        size=1,
        location=(x_pos, 0, domino_size[2] / 2)
    )

    domino = bpy.context.active_object
    domino.name = f"Domino_{i}"
    domino.scale = domino_size
    domino.rotation_euler = (0, 0, 0)

    bpy.ops.object.transform_apply(
        location=False,
        rotation=True,
        scale=True
    )

    bpy.ops.rigidbody.object_add(type='ACTIVE')
    domino.rigid_body.mass = 0.1
    domino.rigid_body.friction = 0.5
    domino.rigid_body.restitution = 0.1
    domino.rigid_body.collision_shape = 'BOX'
    domino.rigid_body.use_margin = True
    domino.rigid_body.collision_margin = 0.001
    domino.rigid_body.use_deactivation = False

bpy.context.view_layer.update()

bpy.ops.mesh.primitive_uv_sphere_add(
    radius=0.2,
    segments=16,
    ring_count=8,
    location=(-2.0, 0, 0.2)
)

pusher = bpy.context.active_object

```

```

pusher.name = "Pusher"
pusher.rotation_euler = (0, 0, 0)

bpy.ops.rigidbody.object_add(type='ACTIVE')

pusher.rigid_body.mass = 2.0
pusher.rigid_body.kinematic = True
pusher.rigid_body.collision_shape = 'SPHERE'
pusher.rigid_body.use_margin = True
pusher.rigid_body.collision_margin = 0.001
pusher.rigid_body.friction = 0.5
pusher.rigid_body.restitution = 0.0

pusher.location.x = -2.0
pusher.keyframe_insert(data_path="location", frame=1)
pusher.keyframe_insert(data_path="location", frame=15)

pusher.location.x = 0.15
pusher.keyframe_insert(data_path="location", frame=35)

pusher.location.x = 0.15
pusher.keyframe_insert(data_path="location", frame=250)

bpy.context.view_layer.update()

bpy.ops.object.camera_add(location=(5, -6, 4))
camera = bpy.context.active_object
camera.name = "Camera"
camera.rotation_euler = (1.1, 0, 0.9)
scene.camera = camera

bpy.ops.object.light_add(type='SUN', location=(5, 5, 10))
sun = bpy.context.active_object
sun.name = "Sun"
sun.data.energy = 3.0
sun.rotation_euler = (0.7, 0.2, 0.5)

scene.frame_start = 1
scene.frame_end = 250
scene.frame_set(1)
scene.render.fps = 24

```

3.5 Настройте рендеринг анимации, используя параметры из предыдущего задания:

- движок рендеринга – **Eevee** для более быстрого выполнения или **Cycles** с 32 выборками;
- разрешение – на усмотрение обучающегося;

- формат видео – **MPEG-4 / H.264**;
- диапазон кадров – от 1 до 250.

3.6 Выполните рендеринг анимации с помощью сочетания клавиш Ctrl + F12. Сохраните видеофайл под именем `domino_experiment.mp4`.

Контрольные вопросы:

Обучающийся должен представить:

1. Какой модуль Python используется для доступа к объектам сцены в Blender?
2. Какое свойство объекта отвечает за назначение физических параметров твёрдого тела?
3. Чем отличается активный тип твёрдого тела (Active) от пассивного (Passive)?
4. Какая команда используется для создания ключевого кадра через Python?
5. Какой параметр отвечает за упругость при столкновениях и в каком диапазоне он изменяется?
6. Для чего устанавливают параметр *substeps_per_frame* и как он влияет на точность симуляции?
7. Как сделать объект кинематическим, то есть управляемым анимацией, но участвующим в физической симуляции?
8. Какой оператор используется для программного удаления всех объектов со сцены?

4 НАСТРОЙКА РЕНДЕРИНГА И ЭКСПОРТ АНИМАЦИИ

4.1 Настройка выборки

В разделе «Выборка» (*Sampling*) установите следующие параметры:

- значение «Рендер» (*Render*) – 32 или 64;
- значение «Видовой экран» (*Viewport*) – 16;
- включите параметр «Шумоподавление» (*Denoise*).

Для видеокарт NVIDIA можно выбрать метод шумоподавления *OptiX*. В качестве универсального варианта используйте *OpenImageDenoise*. Использование шумоподавления позволяет уменьшить количество выборок без существенной потери качества изображения.

4.2 Настройка путей лучей

В разделе «Пути лучей» (*Light Paths*) выполните следующие действия:

- установите значение параметра «Максимальное количество отскоков» (*Max Bounces*) – 4 или 6;
- отключите отражающую и преломляющую каустику, если эти эффекты не являются важными для сцены.

4.3 Настройка производительности

В разделе «Производительность» рекомендуется включить параметр «Постоянные данные» (*Persistent Data*). Эта настройка может сократить время подготовки к повторному рендерингу, если геометрия объектов в сцене не изменяется.

4.4 Настройка экспорта анимации

Перейдите на вкладку «Свойства вывода» (значок принтера на правой панели).

1. В разделе «Диапазон кадров» установите значения: «Начало» – 1, «Конец» – 240 либо укажите собственный диапазон кадров анимации.
2. В разделе «Формат файла» выберите «Видео FFmpeg».
3. Откройте раздел «Кодирование» и установите следующие параметры:
 - контейнер – *MPEG-4*;

- видеокодек – *H.264*;
- качество вывода – «Среднее качество».

При необходимости можно выбрать режим постоянного битрейта и задать значение 4000–6000 кбит/с.

4. Установите разрешение 1280×720 либо уменьшите масштаб вывода до 50%, если производительность компьютера недостаточна для рендеринга в более высоком разрешении.

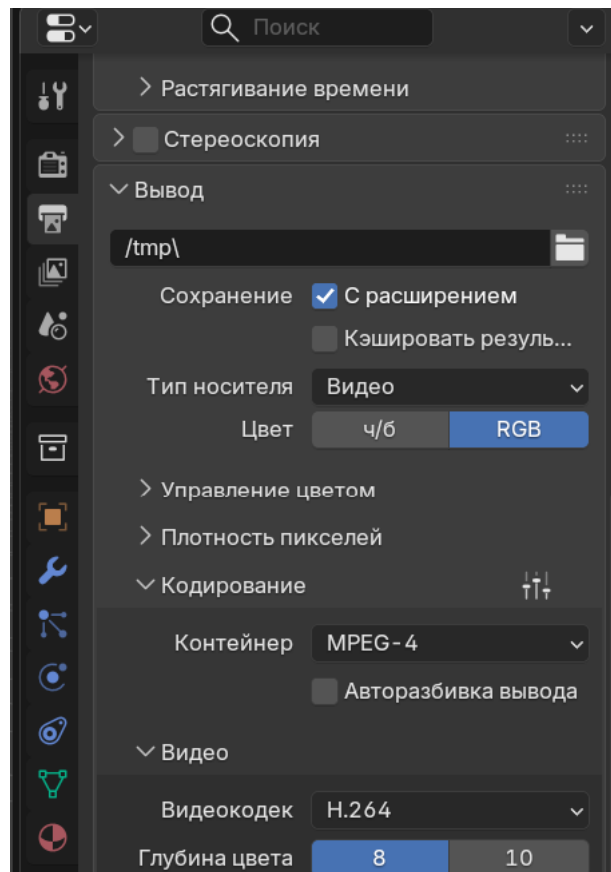


Рис. 2. Настройки выходного формата для экспорта анимации в видеофайл.

4.5 Запуск рендеринга

- Нажмите сочетание клавиш **Ctrl + F12** или выберите команду **Рендер → Рендер анимации**.
- Контролируйте выполнение рендеринга в открывшемся окне.
- Не закрывайте Blender до завершения рендеринга и кодирования видеофайла.

5 КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Какой движок рендеринга используется в данном задании и в чём заключается его главная особенность?
2. Какие устройства вычислений можно выбрать для рендеринга в Cycles?
3. Какую технологию вычислений следует выбрать пользователю с видеокартой NVIDIA? Какую – пользователю с видеокартой AMD?
4. Назовите три способа оптимизации рендеринга в Cycles для компьютеров с ограниченной производительностью.
5. Какие контейнер и видеокодек рекомендуется выбрать для экспорта анимации в готовый видеофайл?